

Ajustement de surface de potentiels

KADDOURI Amyne

Université de Rennes
Master 1 Calcul Scientifique et Modélisation

20 Juillet 2026

Plan de la présentation

Introduction & Contexte

Implémentation du modèle

Méthodes pour les fits de surface

Premiers ajustements de surface

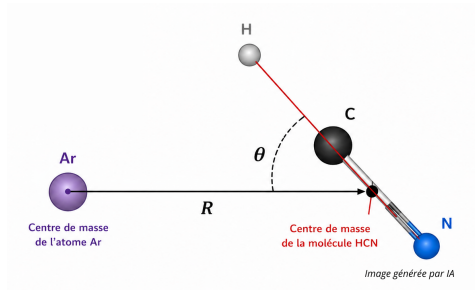
Résultats & Analyse

Conclusion & Perspectives

Contexte théorique

- **Système étudié** : Décrit par une fonction d'onde $\Psi(\vec{r}, \vec{\rho}) = F(\vec{r}) \chi(\vec{\rho}; \vec{r})$
- **Fonction d'onde** : Solution de l'équation de Schrödinger avec
 - $\vec{r}$: position relative Atome - Molécule
 - $\vec{\rho}$: coordonnées internes de l'atome et de la molécule (électrons, etc.)

Figure – Système Ar-HCN



Références

- Theoretical study of the He–HCN, Ne–HCN, Ar–HCN, and Kr–HCN complexes - 2000 - Rafał R. Toczyłowski, Fred Doloresco, and Sławomir M. Cybulski
- Autre article de Toczyłowski avec des paramètres pour chaque systèmes

Objectifs

- Calculer de nouveaux paramètres
- Déterminer comment évaluer leur qualité
- Comparaison des résultats

$$V(R, \theta) = V_{sh}(R, \theta) + V_{as}(R, \theta)$$

Partie courte portée V_{sh}

$$V_{sh} = G(R, \theta) e^{D(\theta) + B(\theta)R}$$

$$G(R, \theta) = \sum_{l=0}^5 (g_{0,l} + g_{1,l}R + g_{2,l}R^2 + g_{3,l}R^3) P_l(\cos \theta)$$

$$D(\theta) = \sum_{l=0}^5 d_l P_l(\cos \theta) \quad B(\theta) = \sum_{l=0}^5 b_l P_l(\cos \theta)$$

Partie asymptotique V_{as}

$$V_{as} = f_6(|B(\theta)R|) \frac{c_0 P_0(\cos \theta) + c_2 P_2(\cos \theta)}{R^6} + f_7(|B(\theta)R|) \frac{c_1 P_1(\cos \theta) + c_3 P_3(\cos \theta)}{R^7}$$

$$f_n(x) = 1 - e^{-x} \sum_{k=0}^n \frac{x^k}{k!}$$

Etapes

- Numérisation des tableaux de données
- Programmation de $V(R, \theta)$ utilisable et fonctionnelle
- Optimisation de V pour de meilleures performances

Bibliothèques et fonctions utilisées

- `numpy / math : cos, exp, radians, abs, ...`
- `scipy.special : eval_legendre, factorial`
- `numpy.polynomial.legendre : legval`

Implémentation plus performante

Points d'amélioration de V

- Vectorisation numpy
- Suppression de calculs inutiles
- Utilisation de fonctions Python plus performantes (ex : `legval`)

Tests des fonctions sur 500 valeurs de R et θ aléatoires

Table – Différence $|V_{init} - V_{opt}|$ (en mEh)

Moyennne	Max	e-type
1.22e-08	1.79e-06	1.09e-07

Table – Temps d'exécution moyen par appel

	V_{init}	V_{opt}
100 appels	3.12e-03s	8.05e-04s
50 least_squares	12.652s	5.759s

Méthodes utilisées pour les ajustements de surface

Fonctions de la bibliothèque `scipy.optimize`

`curve_fit`

- Ajustement de courbes
- On entre des données (x,y)
- Cherche les paramètres qui approchent le mieux ces données
- Converge vers le minimum le plus proche de p0

`least_squares`

- Plus générale que `curve_fit`
- On fixe une fonction résidus
- Choix de bornes possible
- Converge vers le minimum le plus proche de p0

`differential_evolution`

- Algorithme d'optimisation global
- Ne nécessite pas de points de départ
- Méthode stochastique
- Echappe aux minimas locaux
- Plus lente

- He-HCN : 138 points
- Ne-HCN : 151 points
- Ar-HCN : 140 points

Figure –

- Plusieurs ajustements effectués avec différents paramètres d'entrée dans les fonctions
- Première selection graphique
- Deuxième selection à l'aide des métriques : R^2 , RMSE, MAE

Algorithme et Code Python (NumPy & SciPy)

L'algorithme utilise la vectorisation de **NumPy** pour éviter les boucles lentes et la méthode d'intégration de Simpson via **SciPy**.

```
1 import numpy as np
2 from scipy.integrate import simpson
3
4 def compute_v_lambda(r, theta_grid, V_grid, lambda):
5     """
6     Calcule le coefficient radial v_lambda(r) par quadrature numerique.
7     """
8     # Evaluation des polynomes de Legendre sur la grille angulaire
9     P_l = np.polynomial.legendre.legval(np.cos(theta_grid), [0]*lambda + [1])
10
11     # Construction de l'integrande : V(r, theta) * P_lambda(cos(theta)) * sin(theta)
12     integrand = V_grid * P_l * np.sin(theta_grid)
13
14     # Integration par la methode de Simpson
15     factor = (2 * lambda + 1) / 2.0
16     v_l = factor * simpson(y=integrand, x=theta_grid)
17     return v_l
18
```

Analyse d'une Surface d'Énergie Potentielle (PES)

Observations clés :

- Présence d'un puits de potentiel prononcé à la distance d'équilibre r_{eq} .
- Précision de la projection de Legendre validée ($\epsilon < 10^{-4} \text{ cm}^{-1}$).
- **Performance** : Gain d'un facteur 50 par rapport à une implémentation Python pure.

Validation

Les résultats de l'interpolation s'accordent parfaitement avec les données de référence de la littérature.

$V(r) (\text{cm}^{-1})$

Bilan des Réalisations

- **Fiabilité** : Conception d'une méthodologie d'interpolation spectrale performante.
- **Efficacité** : Optimisation de code Python pour le traitement de volumes de données massifs.
- **Outils** : Maîtrise d'un workflow Linux / Python Scientifique / Git.

Perspectives de Recherche

- Modélisation en 3D incluant des vibrations intra-moléculaires.
- Couplage direct de la PES optimisée avec des codes de collision quantique (méthode Close-Coupling).

Merci de votre attention